AfyaHive Frontend Hive

Complete code and architecture guide

This reference documents the maintained Flutter source in frontend_hive. Each implementation file is paired with an explanation of why it exists, how it is named, and how its logic works. Generated platform files and binary images are catalogued by purpose instead of copied verbatim.

Scope: frontend_hive only | Generated on 20 August 2026

1. Folder architecture and naming

The project follows a feature-first Flutter structure. Code is grouped by the user capability it serves, while universal concerns live in core.

Code / file responsibility	Explanation and logic
lib/app/	Application composition. Named app because it contains the root widget and app-wide wiring, rather than a domain feature.
lib/core/	Cross-cutting building blocks: network, theme, and UI primitives. Named core because feature modules depend on it, but it does not depend on features except the authenticated client's session store.
lib/features/	Business-facing modules grouped by capability: auth, home, vitals, settings, shell, and common.
features/*/data/	Data-access classes and models. The name separates HTTP/storage concerns from widgets.
features/*/presentation/	Screens and widgets. The name makes UI responsibility explicit and keeps it separate from data handling.
assets/images/	Bundled visual brand assets referenced by Image.asset.
android/, ios/, linux/, macos/, windows/, web/	Flutter platform runners and native build configuration. These folders are required for their targets; Runner is native host code, not disposable application cache.
test/	Automated tests, isolated from production source.

Reading the naming conventions

snake_case is used for Dart filenames because it is the Dart convention. Widget classes use PascalCase. A leading underscore (for example _AddVital) means the class or member is library-private: it is deliberately reusable only inside that file. presentation identifies UI code; data identifies communication/storage code; common holds generic modules intentionally shared by more than one healthcare feature.

2. Runtime flow

Launch -> main.dart -> AfyaHiveApp -> AuthGate -> LoginScreen or AppShell. Authenticated feature screens call ApiClient -> PHP API -> JSON. Success returns data; failure becomes ApiException -> feature screen message/retry UI.

Code / file responsibility	Explanation and logic
`main() -> runApp(AfyaHiveApp)`	Bootstraps Flutter and installs the root widget.
`AfyaHiveApp -> MaterialApp(home: AuthGate)`	Applies the global theme and delegates the first navigation decision to authentication.
`AuthGate -> secure token -> LoginScreen | AppShell`	Prevents protected views being selected before local session state is known.
`Screen -> ApiClient -> api.php?route=...`	Feature UI asks one shared client to send the token and decode the backend’s response contract.
`{ success, message, data }`	The backend response shape lets all screens distinguish an accepted request from an error consistently.

3. Source-by-source code reference

The left column identifies the file and representative logic; the right column explains its behavior. The complete text of every maintained Dart source file is included in the appendix.

analysis_options.yaml

```
# This file configures the analyzer, which statically an
alyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in
the UI of Dart-enabled
# IDEs (https://dart.dev/tools#ides-and-editors). The an
alyzer can also be
# invoked from the command line by running `flutter anal
yze`.

# The following line activates a set of recommended lint
s for Flutter apps,
# packages, and plugins designed to encourage good codin
g practices.
include: package:flutter_lints/flutter.yaml

linter:
  # The lint rules applied to this project can be custom
ized in the
  # section below to disable rules from the `package:flu
tter_lints/flutter.yaml`
  # included above or to enable additional rules. A list
 of all available lints
  # and their documentation is published at https://dart
.dev/lints.
...
```

Includes Flutter’s recommended lint rules so the analyzer can flag correctness, style, and maintainability concerns.

lib/app/afya_hive_app.dart

```
import 'package:flutter/material.dart';

import '../core/theme/app_theme.dart';
import '../features/auth/presentation/auth_gate.dart';

class AfyaHiveApp extends StatelessWidget {
  const AfyaHiveApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'AfyaHive',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light(),
      darkTheme: AppTheme.dark(),
      themeMode: ThemeMode.system,
...
```

Defines MaterialApp once: product title, system light/dark theme selection, disabled debug banner, and AuthGate as the initial route decision.

lib/core/network/api_client.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../../features/auth/data/auth_session_store.dart
';
import 'api_config.dart';
import 'api_exception.dart';

class ApiClient {
  ApiClient({http.Client? client, AuthSessionStore? sess
ionStore})
    : _client = client ?? http.Client(),
      _sessionStore = sessionStore ?? const AuthSessionS
tore();

  final http.Client _client;
  final AuthSessionStore _sessionStore;

...
```

Adds JSON and bearer-token headers, enforces a 20-second timeout, validates both status and the backend success field, returns only data, and turns malformed/network failures into ApiException.

lib/core/network/api_config.dart

```
import 'package:flutter/foundation.dart';

abstract final class ApiConfig {
  static const _configuredBaseUrl = String.fromEnvironme
nt('API_BASE_URL');

  static String get baseUrl {
    if (_configuredBaseUrl.isNotEmpty) return _configure
dBaseUrl;
    if (kIsWeb) return 'http://localhost/afyahive';
    return defaultTargetPlatform == TargetPlatform.andro
id
        ? 'http://10.0.2.2/afyahive'
        : 'http://127.0.0.1/afyahive';
  }
}
```

Centralizes the backend address. A compile-time API_BASE_URL takes priority. Web uses localhost; Android emulator uses 10.0.2.2 to reach the host computer; desktop uses 127.0.0.1.

lib/core/network/api_exception.dart

```
class ApiException implements Exception {
  const ApiException(this.message, {this.statusCode});

  final String message;
  final int? statusCode;

  @override
  String toString() => message;
}
```

Stores a human-readable message and optional HTTP status. UI code can catch this type and show a safe message without exposing transport details.

lib/core/theme/app_colors.dart

```
import 'package:flutter/material.dart';

abstract final class AppColors {
  static const primary = Color(0xFFE8A515);
  static const primaryDark = Color(0xFFC87C06);
  static const ink = Color.fromARGB(255, 55, 142, 142);
  static const slate = Color.fromARGB(255, 206, 210, 217
);
  static const canvas = Color(0xFFF7F8FA);
  static const surface = Colors.white;
  static const success = Color(0xFF16805C);
  static const danger = Color(0xFFC53B3B);
  static const border = Color(0xFFE4E8EF);
  static const navy = Color(0xFF173A5E);
}
```

Named constants prevent repeated hexadecimal literals and keep health-state colours, surfaces, and brand colours consistent.

lib/core/theme/app_theme.dart

```
import 'package:flutter/material.dart';

import 'app_colors.dart';

abstract final class AppTheme {
  static ThemeData light() => _build(Brightness.light);

  static ThemeData dark() => _build(Brightness.dark);

  static ThemeData _build(Brightness brightness) {
    final isDark = brightness == Brightness.dark;

    final scheme =
        ColorScheme.fromSeed(
          seedColor: AppColors.primary,
          brightness: brightness,
...
```

Builds light and dark ThemeData from the brand seed. It standardizes form fields, buttons, navigation, cards, app bars, and snackbars so feature screens do not restyle every control.

lib/core/ui/app_card.dart

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';

class AppCard extends StatelessWidget {
  const AppCard({
    super.key,
    required this.child,
    this.padding,
    this.onTap,
    this.color,
  });

  final Widget child;
  final EdgeInsetsGeometry? padding;
  final VoidCallback? onTap;
...
```

Applies one card visual style. When onTap is supplied it wraps the card in Material and InkWell, providing ripple/keyboard semantics while non-interactive cards remain lightweight.

lib/core/ui/primary_button.dart

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';

class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.icon,
  });

  final String label;
  final VoidCallback? onPressed;
  final IconData? icon;

...
```

Uses a full-width FilledButton and accepts null onPressed for Flutter’s disabled state. Screens use this to prevent duplicate submissions during loading.

lib/core/ui/section_header.dart

```
import 'package:flutter/material.dart';

class SectionHeader extends StatelessWidget {
  const SectionHeader({
    super.key,
    required this.title,
    this.actionLabel,
    this.onAction,
  });
  final String title;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) => Row(
    children: [
...
```

Renders a title with an optional trailing action. Expanded gives the title remaining width while the action stays aligned.

lib/features/auth/data/auth_repository.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../../../core/network/api_config.dart';
import '../../../core/network/api_exception.dart';

class AuthRepository {
  AuthRepository({http.Client? client}) : _client = clie
nt ?? http.Client();

  final http.Client _client;

  Future<AuthSession> login({
    required String email,
    required String password,
  }) async {
...
```

Owns unauthenticated login/register requests and converts the JSON response into typed AuthSession and AuthUser models. Login and register share transport/error rules.

lib/features/auth/data/auth_session_store.dart

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AuthSessionStore {
  const AuthSessionStore();

  static const _tokenKey = 'afyahive_access_token';
  static const _storage = FlutterSecureStorage();

  Future<void> saveToken(String token) =>
      _storage.write(key: _tokenKey, value: token);
  Future<String?> readToken() => _storage.read(key: _tokenKey);
  Future<void> clear() => _storage.delete(key: _tokenKey);
}
```

Stores only the access token with flutter_secure_storage rather than ordinary preferences. The key is namespaced to the product.

lib/features/auth/presentation/auth_gate.dart

```
import 'package:flutter/material.dart';

import '../../shell/presentation/app_shell.dart';
import '../data/auth_session_store.dart';
import 'login_screen.dart';

class AuthGate extends StatefulWidget {
  const AuthGate({super.key});

  @override
  State<AuthGate> createState() => _AuthGateState();
}

class _AuthGateState extends State<AuthGate> {
  final _sessionStore = const AuthSessionStore();
  late final Future<String?> _token = _sessionStore.readToken();
...
```

Reads the stored token once and shows a progress state until complete. It then selects LoginScreen or AppShell before protected UI is displayed.

lib/features/auth/presentation/login_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/ui/primary_button.dart';
import '../data/auth_repository.dart';
import '../data/auth_session_store.dart';
import 'register_screen.dart';
import '../../shell/presentation/app_shell.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

...
```

Owns form controllers, validators, password visibility, remember-me state, loading lock, API call, token saving, error snackbar, and replacement navigation to the protected shell.

lib/features/auth/presentation/register_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/ui/primary_button.dart';
import '../../shell/presentation/app_shell.dart';
import '../data/auth_repository.dart';
import '../data/auth_session_store.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});
  @override
  State<RegisterScreen> createState() => _RegisterScreen
State();
}

class _RegisterScreenState extends State<RegisterScreen>
 {
  final _form = GlobalKey<FormState>();
  final _first = TextEditingController();
...
```

Validates names, email, password/confirmation; disables repeat submission; registers through AuthRepository; persists the returned token; and replaces the route with AppShell.

lib/features/common/presentation/resource_list_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/primary_button.dart';

enum InputKind { text, number, select, multiline, toggle
 }

class ResourceField {
  const ResourceField(
    this.key,
    this.label, {
    this.kind = InputKind.text,
    this.required = false,
...
```

A configurable feature screen for list/create/delete API resources. ResourceField describes each form control, InputKind selects input behavior, and the screen handles load, validation, submit lock, confirmation, refresh, and error states.

lib/features/common/presentation/service_catalog.dart

```
import 'package:flutter/material.dart';

import 'resource_list_screen.dart';

class ServiceCatalog {
  static void open(BuildContext context, String name) {
    final page = switch (name) {
      'Book an appointment' => const ResourceListScreen(
        title: 'Appointments',
        route: 'appointments',
        emptyMessage: 'No appointments yet.',
        icon: Icons.calendar_month_outlined,
        fields: [
          ResourceField('provider_name', 'Provider name'
, required: true),
          ResourceField('specialty', 'Specialty'),
          ResourceField(
...
```

Maps AfyaHive healthcare modules to title, icon, backend route, empty-state text, and fields. Keeping this metadata separate avoids copy-pasting similar service screens.

lib/features/home/presentation/home_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/section_header.dart';
import '../../common/presentation/service_catalog.dart';
import '../../common/presentation/resource_list_screen.d
art';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  static const _services = [
    (Icons.calendar_month_outlined, 'Book an\nappointmen
t', Color(0xFFEAF8F2)),
    (Icons.medication_outlined, 'Medicine\nreminders', C
olor(0xFFFFF7DE)),
    (Icons.video_camera_front_outlined, 'Telemedicine',
Color(0xFFF0ECFF)),
    (Icons.folder_shared_outlined, 'Medical\nrecords', C
olor(0xFFEAF5F7)),
...
```

Loads dashboard content via ApiClient, shows loading/error/retry UI, and routes feature cards to their existing modules. It remains a presentation layer rather than duplicating API logic.

lib/features/settings/presentation/profile_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/ui/primary_button.dart';

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});
  @override
  State<ProfileScreen> createState() => _ProfileScreenSt
ate();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _api = ApiClient();
  final _form = GlobalKey<FormState>();
  final _date = TextEditingController();
...
```

Loads profile data into disposable controllers, validates required fields, PATCHes the profile, prevents duplicate saves, and supplies loading/error feedback.

lib/features/settings/presentation/settings_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/network/api_client.dart';
import '../../../core/ui/app_card.dart';
import '../../auth/data/auth_session_store.dart';
import '../../auth/presentation/login_screen.dart';
import 'profile_screen.dart';

class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});
  @override
  Widget build(BuildContext context) => ListView(
    padding: const EdgeInsets.fromLTRB(20, 12, 20, 32),
    children: [
      AppCard(
...
```

Provides profile navigation, support/privacy actions, and logout confirmation. It does not store business data itself.

lib/features/shell/presentation/app_shell.dart

```
import 'package:flutter/material.dart';

import '../../home/presentation/home_screen.dart';
import '../../settings/presentation/settings_screen.dart
';
import '../../vitals/presentation/vitals_screen.dart';
import '../../common/presentation/resource_list_screen.d
art';

class AppShell extends StatefulWidget {
  const AppShell({super.key});
  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  var _index = 0;
  static const _pages = [HomeScreen(), VitalsScreen(), S
ettingsScreen()];
...
```

Keeps tab index state and displays the selected feature screen with persistent navigation. Logout clears the secure token after confirmation and returns to login.

lib/features/vitals/presentation/vitals_screen.dart

```
import 'package:flutter/material.dart';
import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/primary_button.dart';
import '../../../core/ui/section_header.dart';

class VitalsScreen extends StatefulWidget {
  const VitalsScreen({super.key});
  @override
  State<VitalsScreen> createState() => _VitalsScreenStat
e();
}

class _VitalsScreenState extends State<VitalsScreen> {
  final _api = ApiClient();
...
```

Loads readings for a selected range, finds the latest record per vital type, opens a preselected entry sheet from each card, validates numeric input, posts manual readings, refreshes after saving, and derives visual status from the Vital model.

lib/main.dart

```
import 'package:flutter/material.dart';

import 'app/afya_hive_app.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const AfyaHiveApp());
}
```

Initializes Flutter bindings before plugins such as secure storage are used, then mounts the root AfyaHiveApp widget.

pubspec.yaml

```
name: frontend_hive
description: "A new Flutter project."
# The following line prevents the package from being acc
identally published to
# pub.dev using `flutter pub publish`. This is preferred
 for private packages.
publish_to: 'none' # Remove this line if you wish to pub
lish to pub.dev

# The following defines the version and build number for
 your application.
# A version number is three numbers separated by dots, l
ike 1.2.43
# followed by an optional build number separated by a +.
# Both the version and the builder number may be overrid
den in flutter
# build by specifying --build-name and --build-number, r
espectively.
# In Android, build-name is used as versionName while bu
ild-number used as versionCode.
# Read more about Android versioning at https://develope
r.android.com/studio/publish/versioning
# In iOS, build-name is used as CFBundleShortVersionStri
ng while build-number is used as CFBundleVersion.
# Read more about iOS versioning at
# https://developer.apple.com/library/archive/documentat
ion/General/Reference/InfoPlistKeyReference/Articles/Cor
eFoundationKeys.html
...
```

Defines package name, SDK limits, dependencies, asset directories, and Flutter configuration. It is the source of truth for build tooling and packages.

test/widget_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:frontend_hive/features/auth/presentation
/login_screen.dart';

void main() {
  testWidgets('shows the AfyaHive sign-in experience', (
tester) async {
    await tester.pumpWidget(const MaterialApp(home: Logi
nScreen()));

    expect(find.text('Your health,\nall in one place.'),
 findsOneWidget);
    expect(find.text('Secure Sign In'), findsOneWidget);
  });
}
```

Pumps LoginScreen and asserts expected visible content. This is a fast guard that the application UI can be constructed in a test environment.

4. Vitals status logic

The Vital model in vitals_screen.dart is the single place that converts a stored numeric reading into a display value, a user-facing status, and a status colour. These are general screening thresholds, not a diagnosis.

Code / file responsibility	Explanation and logic

`blood_pressure`
`value >= 180 || diastolic >= 120 || value < 90`
`-> Abnormal`
	Blood pressure uses the systolic value as value and the lower number as secondary. Extreme high values or systolic under 90 get the abnormal label.

`blood_pressure`
`value >= 130 || diastolic >= 80`
`-> Above healthy range`
`otherwise -> Within healthy range`
	Readings not extreme but at or over these thresholds are highlighted for awareness. Remaining readings receive the healthy-range label.

`heart_rate`
`value < 60 || value > 100`
`-> Abnormal`
`otherwise -> Normal`
	The current application treats 60-100 bpm as normal for a resting general screening range.

`blood_oxygen`
`< 90 Abnormal; < 95 Above healthy range;`
`>= 95 Normal`
	SpO2 has a middle caution band. The display unit is percent.

`temperature`
`< 35 or >= 38 Abnormal;`
`>= 37.5 Above healthy range;`
`otherwise Normal`
	The display unit is Celsius. The status colour is danger for abnormal, primary-dark for caution, and success for normal/recorded.

`weight -> Recorded`	Weight is stored and displayed but is intentionally not classified as normal or abnormal because a safe range requires individual context such as height, age, pregnancy status, and clinical guidance.

5. Native/platform project files

Flutter generates much of this support code. These files are still required to build for their platforms, so they are documented by role rather than expanded as generic boilerplate.

Code / file responsibility	Explanation and logic
android/	Gradle/Kotlin launcher and Android manifest; configures Android packaging, permissions, app label, and Flutter entry activity.
ios/ and macos/	Xcode projects, Swift app delegates, asset catalogues, launch screens, entitlement and build configuration. Runner is the Apple-side application host.
windows/ and linux/	CMake desktop runner code. It creates the native window, registers Flutter plugins, and embeds the Flutter engine.
web/	HTML bootstrap, web manifest, favicon, and PWA icons used by browser builds.
assets/images/	The AfyaHive logo and healthcare banner. Binary image assets are described rather than reproduced as source code.

Important: Do not delete a Runner folder if you intend to build that platform. The Runner project is the platform host. Build artefacts such as build/ and platform flutter/ephemeral/ output can be regenerated.

Appendix A. Complete maintained Dart source

This appendix reproduces the handwritten application source under lib/ and the widget test. It intentionally excludes generated plugin registrants and platform boilerplate; their role is documented above.

lib/app/afya_hive_app.dart

```
import 'package:flutter/material.dart';

import '../core/theme/app_theme.dart';
import '../features/auth/presentation/auth_gate.dart';

class AfyaHiveApp extends StatelessWidget {
  const AfyaHiveApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'AfyaHive',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light(),
      darkTheme: AppTheme.dark(),
      themeMode: ThemeMode.system,
      home: const AuthGate(),
    );
  }
}
```

lib/core/network/api_client.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../../features/auth/data/auth_session_store.dart';
import 'api_config.dart';
import 'api_exception.dart';

class ApiClient {
  ApiClient({http.Client? client, AuthSessionStore? sessionStore})
    : _client = client ?? http.Client(),
      _sessionStore = sessionStore ?? const AuthSessionStore();

  final http.Client _client;
  final AuthSessionStore _sessionStore;

  Future<dynamic> get(String route) => _request('GET', route);
  Future<dynamic> post(String route, Map<String, dynamic> body) =>
      _request('POST', route, body: body);
  Future<dynamic> patch(String route, Map<String, dynamic> body) =>
      _request('PATCH', route, body: body);
  Future<dynamic> delete(String route) => _request('DELETE', route);

  Future<dynamic> _request(
    String method,
    String route, {
    Map<String, dynamic>? body,
  }) async {
    final token = await _sessionStore.readToken();
    if (token == null || token.isEmpty) {
      throw const ApiException(
        'Your session has expired. Please sign in again.',
      );
    }
    try {
      final request =
          http.Request(
              method,
              Uri.parse('${ApiConfig.baseUrl}/api.php?route=$route'),
            )
            ..headers.addAll({
              'Accept': 'application/json',
              'Content-Type': 'application/json',
              'Authorization': 'Bearer $token',
            });
      if (body != null) request.body = jsonEncode(body);
      final streamed = await _client
          .send(request)
          .timeout(const Duration(seconds: 20));
      final response = await http.Response.fromStream(streamed);
      final decoded = jsonDecode(response.body) as Map<String, dynamic>;
      if (response.statusCode < 200 ||
          response.statusCode >= 300 ||
          decoded['success'] != true) {
        throw ApiException(
          decoded['message'] as String? ??
              'The request could not be completed.',
          statusCode: response.statusCode,
        );
      }
      return decoded['data'];
    } on FormatException {
      throw const ApiException('The server returned an invalid response.');
    } on ApiException {
      rethrow;
    } catch (_) {
      throw const ApiException(
        'Unable to reach AfyaHive. Check your connection.',
      );
    }
  }
}
```

lib/core/network/api_config.dart

```
import 'package:flutter/foundation.dart';

abstract final class ApiConfig {
  static const _configuredBaseUrl = String.fromEnvironment('API_BASE_URL');

  static String get baseUrl {
    if (_configuredBaseUrl.isNotEmpty) return _configuredBaseUrl;
    if (kIsWeb) return 'http://localhost/afyahive';
    return defaultTargetPlatform == TargetPlatform.android
        ? 'http://10.0.2.2/afyahive'
        : 'http://127.0.0.1/afyahive';
  }
}
```

lib/core/network/api_exception.dart

```
class ApiException implements Exception {
  const ApiException(this.message, {this.statusCode});

  final String message;
  final int? statusCode;

  @override
  String toString() => message;
}
```

lib/core/theme/app_colors.dart

```
import 'package:flutter/material.dart';

abstract final class AppColors {
  static const primary = Color(0xFFE8A515);
  static const primaryDark = Color(0xFFC87C06);
  static const ink = Color.fromARGB(255, 55, 142, 142);
  static const slate = Color.fromARGB(255, 206, 210, 217);
  static const canvas = Color(0xFFF7F8FA);
  static const surface = Colors.white;
  static const success = Color(0xFF16805C);
  static const danger = Color(0xFFC53B3B);
  static const border = Color(0xFFE4E8EF);
  static const navy = Color(0xFF173A5E);
}
```

lib/core/theme/app_theme.dart

```
import 'package:flutter/material.dart';

import 'app_colors.dart';

abstract final class AppTheme {
  static ThemeData light() => _build(Brightness.light);

  static ThemeData dark() => _build(Brightness.dark);

  static ThemeData _build(Brightness brightness) {
    final isDark = brightness == Brightness.dark;

    final scheme =
        ColorScheme.fromSeed(
          seedColor: AppColors.primary,
          brightness: brightness,
        ).copyWith(
          surface: isDark
              ? const Color.fromARGB(255, 43, 109, 150)
              : const Color.fromARGB(255, 170, 67, 67),
          onSurface: isDark
              ? const Color.fromARGB(255, 0, 9, 2)
              : AppColors.ink,
        );

    return ThemeData(
      useMaterial3: true,

      colorScheme: scheme,

      scaffoldBackgroundColor: isDark
          ? const Color.fromRGBO(255, 255, 255, 1)
          : AppColors.canvas,

      fontFamily: 'Roboto',

      textTheme: Typography.material2021().black.apply(
        bodyColor: scheme.onSurface,
        displayColor: scheme.onSurface,
      ),

      appBarTheme: AppBarTheme(
        backgroundColor: const Color.fromARGB(0, 165, 25, 25),
        foregroundColor: scheme.onSurface,
        elevation: 0,
        scrolledUnderElevation: 0,
      ),

      cardTheme: CardThemeData(
        color: scheme.surface,
        elevation: 0,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(22)),
      ),

      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: isDark
            ? const Color.fromARGB(255, 234, 226, 166)
            : Colors.white,

        contentPadding: const EdgeInsets.symmetric(
          horizontal: 18,
          vertical: 18,
```

```
        ),

        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(18),
          borderSide: const BorderSide(color: AppColors.border),
        ),

        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(18),
          borderSide: const BorderSide(color: AppColors.border),
        ),

        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(18),
          borderSide: BorderSide(color: AppColors.primary, width: 2),
        ),
      ),

      elevatedButtonTheme: ElevatedButtonThemeData(
        style: ElevatedButton.styleFrom(
          backgroundColor: AppColors.primary,
          foregroundColor: Colors.white,
          elevation: 0,
          minimumSize: const Size.fromHeight(56),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(18),
          ),
        ),
      ),

      navigationBarTheme: NavigationBarThemeData(
        backgroundColor: scheme.surface,
        indicatorColor: AppColors.primary.withValues(alpha: .18),

        labelTextStyle: WidgetStatePropertyAll(
          TextStyle(
            color: scheme.onSurface,
            fontWeight: FontWeight.w600,
            fontSize: 12,
          ),
        ),
      ),

      floatingActionButtonTheme: const FloatingActionButtonThemeData(
        backgroundColor: AppColors.primary,
        foregroundColor: Colors.white,
      ),

      snackBarTheme: SnackBarThemeData(
        backgroundColor: AppColors.primary,
        contentTextStyle: const TextStyle(color: Colors.white),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),
        behavior: SnackBarBehavior.floating,
      ),
    );
  }
}
```

lib/core/ui/app_card.dart

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';

class AppCard extends StatelessWidget {
  const AppCard({
    super.key,
    required this.child,
    this.padding,
    this.onTap,
    this.color,
  });

  final Widget child;
  final EdgeInsetsGeometry? padding;
  final VoidCallback? onTap;
  final Color? color;

  @override
  Widget build(BuildContext context) {
    final card = Container(
      padding: padding ?? const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: color ?? Theme.of(context).colorScheme.surface,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: AppColors.border.withValues(alpha: .8)),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: .035),
            blurRadius: 18,
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: child,
    );
    return onTap == null
        ? card
        : Material(
            color: Colors.transparent,
            child: InkWell(
              borderRadius: BorderRadius.circular(20),
              onTap: onTap,
              child: card,
            ),
          );
  }
}
```

lib/core/ui/primary_button.dart

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';

class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.icon,
  });

  final String label;
  final VoidCallback? onPressed;
  final IconData? icon;

  @override
  Widget build(BuildContext context) => SizedBox(
    height: 54,
    width: double.infinity,
    child: FilledButton.icon(
      onPressed: onPressed,
      icon: icon == null ? const SizedBox.shrink() : Icon(icon),
      label: Text(label),
      style: FilledButton.styleFrom(
        backgroundColor: AppColors.primary,
        foregroundColor: Colors.white,
        textStyle: const TextStyle(fontWeight: FontWeight.w700, fontSize: 16),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      ),
    ),
  );
}
```

lib/core/ui/section_header.dart

```
import 'package:flutter/material.dart';

class SectionHeader extends StatelessWidget {
  const SectionHeader({
    super.key,
    required this.title,
    this.actionLabel,
    this.onAction,
  });
  final String title;
  final String? actionLabel;
  final VoidCallback? onAction;

  @override
  Widget build(BuildContext context) => Row(
    children: [
      Expanded(
        child: Text(
          title,
          style: Theme.of(
            context,
          ).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.w800),
        ),
      ),
      if (actionLabel != null)
        TextButton(onPressed: onAction, child: Text(actionLabel!)),
    ],
  );
}
```

lib/features/auth/data/auth_repository.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../../../core/network/api_config.dart';
import '../../../core/network/api_exception.dart';

class AuthRepository {
  AuthRepository({http.Client? client}) : _client = client ?? http.Client();

  final http.Client _client;

  Future<AuthSession> login({
    required String email,
    required String password,
  }) async {
    try {
      final response = await _client
          .post(
            Uri.parse('${ApiConfig.baseUrl}/api.php?route=v1/auth/login'),
            headers: const {
              'Accept': 'application/json',
              'Content-Type': 'application/json',
            },
            body: jsonEncode({'email': email, 'password': password}),
          )
          .timeout(const Duration(seconds: 20));
      final decoded = jsonDecode(response.body) as Map<String, dynamic>;
      if (response.statusCode < 200 ||
          response.statusCode >= 300 ||
          decoded['success'] != true) {
        throw ApiException(
          decoded['message'] as String? ?? 'Unable to sign in.',
          statusCode: response.statusCode,
        );
      }
      return AuthSession.fromJson(decoded['data'] as Map<String, dynamic>);
    } on FormatException {
      throw const ApiException('The server returned an invalid response.');
    } on ApiException {
      rethrow;
    } catch (_) {
      throw const ApiException(
```

```
        'Unable to reach AfyaHive. Check your connection.',
      );
    }
  }

  Future<AuthSession> register({
    required String firstname,
    required String lastname,
    required String email,
    required String password,
  }) => _authenticate('register', {
    'firstname': firstname,
    'lastname': lastname,
    'email': email,
    'password': password,
  });

  Future<AuthSession> _authenticate(
    String action,
    Map<String, String> body,
  ) async {
    try {
      final response = await _client
          .post(
            Uri.parse('${ApiConfig.baseUrl}/api.php?route=v1/auth/$action'),
            headers: const {
              'Accept': 'application/json',
              'Content-Type': 'application/json',
            },
            body: jsonEncode(body),
          )
          .timeout(const Duration(seconds: 20));
      final decoded = jsonDecode(response.body) as Map<String, dynamic>;
      if (response.statusCode < 200 ||
          response.statusCode >= 300 ||
          decoded['success'] != true) {
        throw ApiException(
          decoded['message'] as String? ?? 'Unable to continue.',
          statusCode: response.statusCode,
        );
      }
      return AuthSession.fromJson(decoded['data'] as Map<String, dynamic>);
    } on FormatException {
      throw const ApiException('The server returned an invalid response.');
    } on ApiException {
      rethrow;
    } catch (_) {
      throw const ApiException(
        'Unable to reach AfyaHive. Check your connection.',
      );
    }
  }
}

class AuthSession {
  const AuthSession({required this.accessToken, required this.user});

  final String accessToken;
  final AuthUser user;

  factory AuthSession.fromJson(Map<String, dynamic> json) => AuthSession(
    accessToken: json['accessToken'] as String,
    user: AuthUser.fromJson(json['user'] as Map<String, dynamic>),
  );
}

class AuthUser {
  const AuthUser({
    required this.id,
    required this.firstname,
    required this.lastname,
    required this.email,
  });

  final int id;
  final String firstname;
  final String lastname;
  final String email;

  factory AuthUser.fromJson(Map<String, dynamic> json) => AuthUser(
    id: json['id'] as int,
    firstname: json['firstname'] as String,
    lastname: json['lastname'] as String,
    email: json['email'] as String,
  );
}
```

lib/features/auth/data/auth_session_store.dart

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AuthSessionStore {
  const AuthSessionStore();

  static const _tokenKey = 'afyahive_access_token';
  static const _storage = FlutterSecureStorage();

  Future<void> saveToken(String token) =>
      _storage.write(key: _tokenKey, value: token);
  Future<String?> readToken() => _storage.read(key: _tokenKey);
  Future<void> clear() => _storage.delete(key: _tokenKey);
}
```

lib/features/auth/presentation/auth_gate.dart

```
import 'package:flutter/material.dart';

import '../../shell/presentation/app_shell.dart';
import '../data/auth_session_store.dart';
import 'login_screen.dart';
class AuthGate extends StatefulWidget {
```

```
  const AuthGate({super.key});

  @override
  State<AuthGate> createState() => _AuthGateState();
}

class _AuthGateState extends State<AuthGate> {
  final _sessionStore = const AuthSessionStore();
  late final Future<String?> _token = _sessionStore.readToken();

  @override
  Widget build(BuildContext context) => FutureBuilder<String?>(
    future: _token,
    builder: (context, snapshot) {
      if (snapshot.connectionState != ConnectionState.done) {
        return const Scaffold(body: Center(child: CircularProgressIndicator()));
      }
      return snapshot.data == null || snapshot.data!.isEmpty
          ? const LoginScreen()
          : const AppShell();
    },
  );
}
```

lib/features/auth/presentation/login_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/ui/primary_button.dart';
import '../data/auth_repository.dart';
import '../data/auth_session_store.dart';
import 'register_screen.dart';
import '../../shell/presentation/app_shell.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();

  final _email = TextEditingController();
  final _password = TextEditingController();

  bool _obscure = true;
  bool _rememberMe = true;
  bool _isSubmitting = false;
  final _authRepository = AuthRepository();
  final _sessionStore = const AuthSessionStore();

  @override
  void dispose() {
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    if (!_formKey.currentState!.validate() || _isSubmitting) return;
    setState(() => _isSubmitting = true);
    try {
      final session = await _authRepository.login(
        email: _email.text.trim(),
        password: _password.text,
      );
      if (_rememberMe) await _sessionStore.saveToken(session.accessToken);
      if (!mounted) return;
      Navigator.of(
        context,
      ).pushReplacement(MaterialPageRoute(builder: (_) => const AppShell()));
    } catch (error) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(error.toString())));
      }
    } finally {
      if (mounted) setState(() => _isSubmitting = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: LayoutBuilder(
          builder: (context, constraints) => Center(
            child: SingleChildScrollView(
              padding: const EdgeInsets.fromLTRB(24, 28, 24, 32),
              child: ConstrainedBox(
                constraints: const BoxConstraints(maxWidth: 460),
                child: Form(
                  key: _formKey,
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Hero(
                        tag: 'afyahive-logo',
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            CircleAvatar(
                              radius: 30,
                              backgroundColor: AppColors.primary.withValues(
                                alpha: .15,
                              ),
                              child: Image.asset(
                                'assets/images/afyahive_logo.png',
                                fit: BoxFit.contain,
                              ),
```

```
      ),

      const SizedBox(height: 12),

      Text(
        'AfyaHive',
        style: Theme.of(context).textTheme.headlineMedium
            ?.copyWith(
              fontWeight: FontWeight.w900,
              color: AppColors.primary,
              letterSpacing: -0.5,
            ),
      ),

      Text(
        'Healthcare Platform',
        style: Theme.of(context).textTheme.bodyMedium
            ?.copyWith(
              color: Colors.grey.shade600,
              fontWeight: FontWeight.w500,
            ),
      ),
    ],
  ),
),

const SizedBox(height: 28),

Text(
  'Your health,\nall in one place.',
  style: Theme.of(context).textTheme.displaySmall
      ?.copyWith(
        fontWeight: FontWeight.w800,
        height: 1.08,
      ),
),

const SizedBox(height: 10),

Text(
  'Welcome back to AfyaHive. Continue your care journey securely.',
  style: Theme.of(context).textTheme.bodyLarge?.copyWith(
    color: const Color.fromARGB(255, 55, 61, 173),
    height: 1.45,
  ),
),

const SizedBox(height: 32),

TextFormField(
  controller: _email,
  keyboardType: TextInputType.emailAddress,
  autofillHints: const [AutofillHints.email],
  decoration: const InputDecoration(
    labelText: 'Email address',
    prefixIcon: Icon(Icons.mail_outline),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Email is required';
    }

    if (!RegExp(r'^\S+@\S+\.\S+$').hasMatch(value)) {
      return 'Enter a valid email';
    }

    return null;
  },
),

const SizedBox(height: 16),

TextFormField(
  controller: _password,
  obscureText: _obscure,
  autofillHints: const [AutofillHints.password],
  decoration: InputDecoration(
    labelText: 'Password',
    prefixIcon: const Icon(Icons.lock_outline),
    suffixIcon: IconButton(
      icon: Icon(
        _obscure
            ? Icons.visibility_off_outlined
            : Icons.visibility_outlined,
      ),
      onPressed: () {
        setState(() {
          _obscure = !_obscure;
        });
      },
    ),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Password is required';
    }

    return null;
  },
),

Row(
  children: [
    Checkbox(
      value: _rememberMe,
      onChanged: (value) {
        setState(() {
          _rememberMe = value ?? false;
        });
      },
    ),
    const Text('Remember me'),
    const Spacer(),
    TextButton(
      onPressed: () => showDialog<void>(
        context: context,
        builder: (context) => AlertDialog(
```

```
                            title: const Text('Password recovery'),
                            content: const Text(
                              'Password recovery has not been configured for this local development server. Please contact your AfyaHive administrator.',
                            ),
                            actions: [
                              TextButton(
                                onPressed: () => Navigator.pop(context),
                                child: const Text('Close'),
                              ),
                            ],
                          ),
                        ),
                        child: const Text('Forgot password?'),
                      ),
                    ],
                  ),

                  const SizedBox(height: 16),

                  PrimaryButton(
                    label: _isSubmitting
                        ? 'Signing in...'
                        : 'Secure Sign In',
                    icon: Icons.arrow_forward_rounded,
                    onPressed: _isSubmitting ? null : _login,
                  ),

                  const SizedBox(height: 20),

                  Center(
                    child: TextButton(
                      onPressed: () => Navigator.of(context).push(
                        MaterialPageRoute(
                          builder: (_) => const RegisterScreen(),
                        ),
                      ),
                      child: const Text(
                        'New to AfyaHive? Create an account',
                      ),
                    ),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

lib/features/auth/presentation/register_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/ui/primary_button.dart';
import '../../shell/presentation/app_shell.dart';
import '../data/auth_repository.dart';
import '../data/auth_session_store.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});
  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _form = GlobalKey<FormState>();
  final _first = TextEditingController();
  final _last = TextEditingController();
  final _email = TextEditingController();
  final _password = TextEditingController();
  final _repository = AuthRepository();
  bool _saving = false;

  @override
  void dispose() {
    _first.dispose();
    _last.dispose();
    _email.dispose();
    _password.dispose();
    super.dispose();
  }

  Future<void> _register() async {
    if (!_form.currentState!.validate() || _saving) return;
    setState(() => _saving = true);
    try {
      final session = await _repository.register(
        firstname: _first.text.trim(),
        lastname: _last.text.trim(),
        email: _email.text.trim(),
        password: _password.text,
      );
      await const AuthSessionStore().saveToken(session.accessToken);
      if (!mounted) return;
      Navigator.of(context).pushAndRemoveUntil(
        MaterialPageRoute(builder: (_) => const AppShell()),
        (_) => false,
      );
    } catch (error) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(error.toString())));
      }
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  @override
```

```
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Create account')),
    body: SafeArea(
      top: false,
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 460),
            child: Form(
              key: _form,
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'Join AfyaHive',
                    style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                  const SizedBox(height: 8),
                  const Text('Create your secure healthcare account.'),
                  const SizedBox(height: 28),
                  TextFormField(
                    controller: _first,
                    decoration: const InputDecoration(labelText: 'First name'),
                    validator: (v) => v == null || v.trim().isEmpty
                        ? 'First name is required'
                        : null,
                  ),
                  const SizedBox(height: 14),
                  TextFormField(
                    controller: _last,
                    decoration: const InputDecoration(labelText: 'Last name'),
                    validator: (v) => v == null || v.trim().isEmpty
                        ? 'Last name is required'
                        : null,
                  ),
                  const SizedBox(height: 14),
                  TextFormField(
                    controller: _email,
                    keyboardType: TextInputType.emailAddress,
                    decoration: const InputDecoration(
                      labelText: 'Email address',
                    ),
                    validator: (v) =>
                        v != null && RegExp(r'^\S+@\S+\.\S+$').hasMatch(v)
                        ? null
                        : 'Enter a valid email',
                  ),
                  const SizedBox(height: 14),
                  TextFormField(
                    controller: _password,
                    obscureText: true,
                    decoration: const InputDecoration(labelText: 'Password'),
                    validator: (v) => (v?.length ?? 0) >= 8
                        ? null
                        : 'Use at least 8 characters',
                  ),
                  const SizedBox(height: 24),
                  PrimaryButton(
                    label: _saving ? 'Creating account...' : 'Create account',
                    onPressed: _saving ? null : _register,
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

lib/features/common/presentation/resource_list_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/primary_button.dart';

enum InputKind { text, number, select, multiline, toggle }

class ResourceField {
  const ResourceField(
    this.key,
    this.label, {
    this.kind = InputKind.text,
    this.required = false,
    this.options = const [],
  });
  final String key;
  final String label;
  final InputKind kind;
  final bool required;
  final List<String> options;
}

class ResourceListScreen extends StatefulWidget {
  const ResourceListScreen({
    super.key,
    required this.title,
    required this.route,
    required this.fields,
    required this.emptyMessage,
    this.icon = Icons.health_and_safety_outlined,
  });
  final String title;
  final String route;
  final List<ResourceField> fields;
  final String emptyMessage;
```

```
  final IconData icon;
  @override
  State<ResourceListScreen> createState() => _ResourceListScreenState();
}

class _ResourceListScreenState extends State<ResourceListScreen> {
  final _api = ApiClient();
  late Future<List<Map<String, dynamic>>> _items;
  @override
  void initState() {
    super.initState();
    _items = _load();
  }

  Future<List<Map<String, dynamic>>> _load() async =>
      List<Map<String, dynamic>>.from(
        await _api.get('v1/${widget.route}') as List,
      );
  Future<void> _refresh() async {
    setState(() => _items = _load());
  }

  Future<void> _create() async {
    final created = await showModalBottomSheet<bool>(
      context: context,
      isScrollControlled: true,
      builder: (_) => _CreateSheet(
        title: widget.title,
        fields: widget.fields,
        api: _api,
        route: widget.route,
      ),
    );
    if (!mounted || created != true) return;
    await _refresh();
    if (mounted) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(const SnackBar(content: Text('Saved successfully.')));
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text(widget.title)),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: _create,
      icon: const Icon(Icons.add_rounded),
      label: const Text('Add'),
    ),
    body: FutureBuilder<List<Map<String, dynamic>>>(
      future: _items,
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return _Error(message: _error(snapshot.error), onRetry: _refresh);
        }
        final items = snapshot.data!;
        if (items.isEmpty) {
          return _Empty(
            icon: widget.icon,
            message: widget.emptyMessage,
            onAdd: _create,
          );
        }
        return RefreshIndicator(
          onRefresh: _refresh,
          child: ListView.separated(
            padding: const EdgeInsets.fromLTRB(20, 16, 20, 100),
            itemCount: items.length,
            separatorBuilder: (_, _) => const SizedBox(height: 12),
            itemBuilder: (context, index) => _Card(
              item: items[index],
              onDelete: () async {
                await _api.delete('v1/${widget.route}/${items[index]['id']}');
                if (!mounted) return;
                await _refresh();
                if (mounted) {
                  ScaffoldMessenger.of(
                    this.context,
                  ).showSnackBar(const SnackBar(content: Text('Deleted.')));
                }
              },
            ),
          ),
        );
      },
    ),
  );
}

class _Card extends StatelessWidget {
  const _Card({required this.item, required this.onDelete});
  final Map<String, dynamic> item;
  final Future<void> Function() onDelete;
  @override
  Widget build(BuildContext context) {
    final title =
        item['medication_name'] ??
        item['provider_name'] ??
        item['workout_name'] ??
        item['goal_type'] ??
        item['name'] ??
        item['title'] ??
        item['body'] ??
        item['activity_type'] ??
        'AfyaHive item';
    final detail = item.entries
        .where(
          (e) => ![
            'id',
            'user_id',
            'created_at',
            'updated_at',
            'title',
```

```
            'body',
          ].contains(e.key),
        )
        .take(2)
        .map((e) => '${e.key.replaceAll('_', ' ')}: ${e.value}')
        .join('\n');
    return AppCard(
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  '$title',
                  maxLines: 2,
                  overflow: TextOverflow.ellipsis,
                  style: const TextStyle(fontWeight: FontWeight.w800),
                ),
                if (detail.isNotEmpty)
                  Padding(
                    padding: const EdgeInsets.only(top: 7),
                    child: Text(
                      detail,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(
                        fontSize: 12,
                        color: AppColors.slate,
                      ),
                    ),
                  ),
              ],
            ),
          ),
          IconButton(
            tooltip: 'Delete',
            icon: const Icon(
              Icons.delete_outline_rounded,
              color: AppColors.danger,
            ),
            onPressed: () async {
              final confirm = await showDialog<bool>(
                context: context,
                builder: (dialogContext) => AlertDialog(
                  title: const Text('Delete item?'),
                  content: const Text('This action cannot be undone.'),
                  actions: [
                    TextButton(
                      onPressed: () => Navigator.pop(dialogContext, false),
                      child: const Text('Cancel'),
                    ),
                    FilledButton(
                      onPressed: () => Navigator.pop(dialogContext, true),
                      child: const Text('Delete'),
                    ),
                  ],
                ),
              );
              if (confirm == true) await onDelete();
            },
          ),
        ],
      ),
    );
  }
}

class _CreateSheet extends StatefulWidget {
  const _CreateSheet({
    required this.title,
    required this.fields,
    required this.api,
    required this.route,
  });
  final String title;
  final List<ResourceField> fields;
  final ApiClient api;
  final String route;
  @override
  State<_CreateSheet> createState() => _CreateSheetState();
}

class _CreateSheetState extends State<_CreateSheet> {
  final _form = GlobalKey<FormState>();
  final _controllers = <String, TextEditingController>{};
  final _values = <String, dynamic>{};
  bool _saving = false;
  @override
  void initState() {
    super.initState();
    for (final field in widget.fields) {
      _controllers[field.key] = TextEditingController();
      if (field.kind == InputKind.select) {
        _values[field.key] = field.options.first;
      }
      if (field.kind == InputKind.toggle) _values[field.key] = true;
    }
  }

  @override
  void dispose() {
    for (final controller in _controllers.values) {
      controller.dispose();
    }
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_form.currentState!.validate() || _saving) return;
    setState(() => _saving = true);
    try {
      final body = <String, dynamic>{};
      for (final field in widget.fields) {
        if (field.kind == InputKind.select || field.kind == InputKind.toggle) {
          body[field.key] = _values[field.key];
```

```
        } else {
          final text = _controllers[field.key]!.text.trim();
          if (text.isNotEmpty) {
            body[field.key] = field.kind == InputKind.number
                ? num.parse(text)
                : text;
          }
        }
      }
      await widget.api.post('v1/${widget.route}', body);
      if (mounted) Navigator.pop(context, true);
    } on ApiException catch (error) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(error.message)));
      }
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  @override
  Widget build(BuildContext context) => Padding(
    padding: EdgeInsets.fromLTRB(
      20,
      20,
      20,
      MediaQuery.viewInsetsOf(context).bottom + 20,
    ),
    child: SafeArea(
      top: false,
      child: SingleChildScrollView(
        child: Form(
          key: _form,
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Add ${widget.title}',
                style: Theme.of(
                  context,
                ).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.w800),
              ),
              const SizedBox(height: 16),
              ...widget.fields.map(_field),
              const SizedBox(height: 12),
              PrimaryButton(
                label: _saving ? 'Saving...' : 'Save',
                onPressed: _saving ? null : _submit,
              ),
            ],
          ),
        ),
      ),
    ),
  );
  Widget _field(ResourceField field) {
    if (field.kind == InputKind.select) {
      return Padding(
        padding: const EdgeInsets.only(bottom: 12),
        child: DropdownButtonFormField<String>(
          initialValue: _values[field.key] as String,
          decoration: InputDecoration(labelText: field.label),
          items: field.options
              .map(
                (option) =>
                    DropdownMenuItem(value: option, child: Text(option)),
              )
              .toList(),
          onChanged: (value) => setState(() => _values[field.key] = value),
        ),
      );
    }
    if (field.kind == InputKind.toggle) {
      return SwitchListTile.adaptive(
        contentPadding: EdgeInsets.zero,
        title: Text(field.label),
        value: _values[field.key] as bool,
        onChanged: (value) => setState(() => _values[field.key] = value),
      );
    }
    return Padding(
      padding: const EdgeInsets.only(bottom: 12),
      child: TextFormField(
        controller: _controllers[field.key],
        keyboardType: field.kind == InputKind.number
            ? const TextInputType.numberWithOptions(decimal: true)
            : field.kind == InputKind.multiline
            ? TextInputType.multiline
            : null,
        maxLines: field.kind == InputKind.multiline ? 3 : 1,
        decoration: InputDecoration(labelText: field.label),
        validator: field.required
            ? (value) => value == null || value.trim().isEmpty
                  ? '${field.label} is required'
                  : null
            : null,
      ),
    );
  }
}

class _Empty extends StatelessWidget {
  const _Empty({
    required this.icon,
    required this.message,
    required this.onAdd,
  });
  final IconData icon;
  final String message;
  final VoidCallback onAdd;
  @override
  Widget build(BuildContext context) => Center(
    child: Padding(
      padding: const EdgeInsets.all(32),
```

```
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(icon, size: 52, color: AppColors.primary),
          const SizedBox(height: 16),
          Text(message, textAlign: TextAlign.center),
          const SizedBox(height: 18),
          FilledButton.icon(
            onPressed: onAdd,
            icon: const Icon(Icons.add),
            label: const Text('Add now'),
          ),
        ],
      ),
    ),
  );
}

class _Error extends StatelessWidget {
  const _Error({required this.message, required this.onRetry});
  final String message;
  final Future<void> Function() onRetry;
  @override
  Widget build(BuildContext context) => Center(
    child: Padding(
      padding: const EdgeInsets.all(32),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const Icon(
            Icons.cloud_off_rounded,
            size: 48,
            color: AppColors.danger,
          ),
          const SizedBox(height: 12),
          Text(message, textAlign: TextAlign.center),
          const SizedBox(height: 12),
          OutlinedButton(onPressed: onRetry, child: const Text('Try again')),
        ],
      ),
    ),
  );
}

String _error(Object? error) =>
    error is ApiException ? error.message : 'Unable to load data.';
```

lib/features/common/presentation/service_catalog.dart

```
import 'package:flutter/material.dart';

import 'resource_list_screen.dart';

class ServiceCatalog {
  static void open(BuildContext context, String name) {
    final page = switch (name) {
      'Book an appointment' => const ResourceListScreen(
        title: 'Appointments',
        route: 'appointments',
        emptyMessage: 'No appointments yet.',
        icon: Icons.calendar_month_outlined,
        fields: [
          ResourceField('provider_name', 'Provider name', required: true),
          ResourceField('specialty', 'Specialty'),
          ResourceField(
            'scheduled_at',
            'Scheduled at (YYYY-MM-DD HH:MM:SS)',
            required: true,
          ),
          ResourceField('location', 'Location'),
          ResourceField(
            'mode',
            'Visit type',
            kind: InputKind.select,
            options: ['in_person', 'telemedicine'],
          ),
          ResourceField('notes', 'Notes', kind: InputKind.multiline),
        ],
      ),
      'Medicine reminders' => const ResourceListScreen(
        title: 'Medicine reminders',
        route: 'reminders',
        emptyMessage: 'No medicine reminders yet.',
        icon: Icons.medication_outlined,
        fields: [
          ResourceField('medication_name', 'Medicine name', required: true),
          ResourceField('dosage', 'Dosage'),
          ResourceField('schedule_time', 'Time (HH:MM:SS)', required: true),
          ResourceField(
            'frequency',
            'Frequency',
            kind: InputKind.select,
            options: ['daily', 'weekly', 'as_needed'],
          ),
          ResourceField('is_active', 'Reminder active', kind: InputKind.toggle),
          ResourceField('notes', 'Notes', kind: InputKind.multiline),
        ],
      ),
      'Telemedicine' => const ResourceListScreen(
        title: 'Telemedicine',
        route: 'telemedicine',
        emptyMessage: 'No telemedicine sessions yet.',
        icon: Icons.video_camera_front_outlined,
        fields: [
          ResourceField('provider_name', 'Provider name', required: true),
          ResourceField(
            'scheduled_at',
            'Scheduled at (YYYY-MM-DD HH:MM:SS)',
            required: true,
          ),
          ResourceField('meeting_url', 'Meeting URL'),
          ResourceField(
            'status',
            'Status',
```

```
            kind: InputKind.select,
            options: ['scheduled', 'completed', 'cancelled'],
          ),
        ],
      ),
      'Medical records' => const ResourceListScreen(
        title: 'Medical records',
        route: 'medical-records',
        emptyMessage: 'No medical records yet.',
        icon: Icons.folder_shared_outlined,
        fields: [
          ResourceField(
            'record_type',
            'Record type',
            required: true,
            kind: InputKind.select,
            options: ['lab_result', 'prescription', 'diagnosis', 'document'],
          ),
          ResourceField('title', 'Title', required: true),
          ResourceField('details', 'Details', kind: InputKind.multiline),
          ResourceField('issued_at', 'Issued date (YYYY-MM-DD)'),
          ResourceField('file_url', 'Secure file URL'),
        ],
      ),
      'Health community' => const ResourceListScreen(
        title: 'Health community',
        route: 'community-posts',
        emptyMessage: 'No community posts yet.',
        icon: Icons.groups_2_outlined,
        fields: [
          ResourceField(
            'body',
            'Share an update',
            required: true,
            kind: InputKind.multiline,
          ),
          ResourceField(
            'is_anonymous',
            'Post anonymously',
            kind: InputKind.toggle,
          ),
        ],
      ),
      'Emergency contacts' => const ResourceListScreen(
        title: 'Emergency contacts',
        route: 'emergency-contacts',
        emptyMessage: 'Add a trusted contact for emergencies.',
        icon: Icons.emergency_outlined,
        fields: [
          ResourceField('name', 'Full name', required: true),
          ResourceField('relationship', 'Relationship', required: true),
          ResourceField('phone', 'Phone number', required: true),
          ResourceField(
            'is_primary',
            'Primary contact',
            kind: InputKind.toggle,
          ),
        ],
      ),
      'Fitness integration' => const ResourceListScreen(
        title: 'Activity tracking',
        route: 'activities',
        emptyMessage: 'No activity has been logged yet.',
        icon: Icons.directions_walk_rounded,
        fields: [
          ResourceField(
            'activity_type',
            'Activity type',
            required: true,
            kind: InputKind.select,
            options: [
              'Walking',
              'Running',
              'Cycling',
              'Swimming',
              'Yoga',
              'Other',
            ],
          ),
          ResourceField(
            'duration_minutes',
            'Duration in minutes',
            required: true,
            kind: InputKind.number,
          ),
          ResourceField(
            'calories_burned',
            'Calories burned',
            required: true,
            kind: InputKind.number,
          ),
          ResourceField('distance_km', 'Distance (km)', kind: InputKind.number),
          ResourceField(
            'activity_date',
            'Activity date (YYYY-MM-DD)',
            required: true,
          ),
          ResourceField('notes', 'Notes', kind: InputKind.multiline),
        ],
      ),
      _ => null,
    };
    if (page == null) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text(
            'This clinical service requires an approved provider integration before it can be used.',
          ),
        ),
      );
      return;
    }
    Navigator.of(context).push(MaterialPageRoute(builder: (_) => page));
  }
}
```

lib/features/home/presentation/home_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/section_header.dart';
import '../../common/presentation/service_catalog.dart';
import '../../common/presentation/resource_list_screen.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  static const _services = [
    (Icons.calendar_month_outlined, 'Book an\nappointment', Color(0xFFEAF8F2)),
    (Icons.medication_outlined, 'Medicine\nreminders', Color(0xFFFFF7DE)),
    (Icons.video_camera_front_outlined, 'Telemedicine', Color(0xFFF0ECFF)),
    (Icons.folder_shared_outlined, 'Medical\nrecords', Color(0xFFEAF5F7)),
    (Icons.groups_2_outlined, 'Health\ncommunity', Color(0xFFFFEDF5)),
    (Icons.emergency_outlined, 'Emergency\ncontacts', Color(0xFFFFEDEC)),
  ];

  @override
  Widget build(BuildContext context) => CustomScrollView(
    key: const PageStorageKey('home-scroll'),
    slivers: [
      SliverPadding(
        padding: const EdgeInsets.fromLTRB(20, 12, 20, 28),
        sliver: SliverList(
          delegate: SliverChildListDelegate([
            Text(
              'Good morning, Jakes',
              style: Theme.of(
                context,
              ).textTheme.headlineSmall?.copyWith(fontWeight: FontWeight.w800),
            ),
            const SizedBox(height: 6),
            Text(
              'A clearer view of your wellbeing today.',
              style: Theme.of(context).textTheme.bodyLarge?.copyWith(
                color: const Color.fromARGB(255, 52, 96, 173),
              ),
            ),
            const SizedBox(height: 20),
            _HealthScoreCard(),
            const SizedBox(height: 28),
            SectionHeader(
              title: 'Care at your fingertips',
              actionLabel: 'See all',
              onAction: () =>
                  ServiceCatalog.open(context, 'Fitness integration'),
            ),
            const SizedBox(height: 12),
          ]),
        ),
      ),
      SliverPadding(
        padding: const EdgeInsets.symmetric(horizontal: 20),
        sliver: SliverGrid(
          gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 4,
            mainAxisSpacing: 14,
            crossAxisSpacing: 10,
            childAspectRatio: .72,
          ),
          delegate: SliverChildBuilderDelegate((context, index) {
            final service = _services[index];
            return Semantics(
              button: true,
              label: service.$2.replaceAll('\n', ' '),
              child: InkWell(
                borderRadius: BorderRadius.circular(16),
                onTap: () => ServiceCatalog.open(
                  context,
                  service.$2.replaceAll('\n', ' '),
                ),
                child: Column(
                  children: [
                    Container(
                      height: 52,
                      width: 52,
                      decoration: BoxDecoration(
                        color: service.$3,
                        borderRadius: BorderRadius.circular(16),
                      ),
                      child: Icon(service.$1, color: AppColors.navy),
                    ),
                    const SizedBox(height: 7),
                    Text(
                      service.$2,
                      textAlign: TextAlign.center,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(
                        fontSize: 11,
                        height: 1.15,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ],
                ),
              ),
            );
          }, childCount: _services.length),
        ),
      ),
      SliverPadding(
        padding: const EdgeInsets.fromLTRB(20, 26, 20, 32),
        sliver: SliverList(
          delegate: SliverChildListDelegate([
            const SectionHeader(title: "Today's care plan"),
            const SizedBox(height: 12),
            AppCard(
              onTap: () => Navigator.of(context).push(
                MaterialPageRoute(
                  builder: (_) => const ResourceListScreen(
```

```
                    title: 'Medicine reminders',
                    route: 'reminders',
                    emptyMessage: 'No medicine reminders yet.',
                    icon: Icons.medication_outlined,
                    fields: [
                      ResourceField(
                        'medication_name',
                        'Medicine name',
                        required: true,
                      ),
                      ResourceField(
                        'schedule_time',
                        'Time (HH:MM:SS)',
                        required: true,
                      ),
                      ResourceField(
                        'frequency',
                        'Frequency',
                        kind: InputKind.select,
                        options: ['daily', 'weekly', 'as_needed'],
                      ),
                    ],
                  ),
                ),
              ),
              child: Column(
                children: [
                  _CarePlanRow(
                    icon: Icons.medication_outlined,
                    color: const Color.fromARGB(255, 224, 185, 185),
                    title: 'Medication reminder',
                    subtitle: 'Medicine to be taken',
                    trailing: 'Pending',
                  ),
                  const Divider(height: 24),
                  _CarePlanRow(
                    icon: Icons.event_available_outlined,
                    color: const Color.fromARGB(255, 153, 233, 206),
                    title: 'Medical appointment',
                    subtitle: 'Routine consultation',
                    trailing: 'Confirmed',
                  ),
                ],
              ),
            ),
            const SizedBox(height: 24),
            const SectionHeader(title: 'Wellness insights'),
            const SizedBox(height: 12),
            const _InsightCard(),
          ]),
        ),
      ),
    ],
  );
}

class _HealthScoreCard extends StatelessWidget {
  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      gradient: const LinearGradient(
        colors: [AppColors.navy, Color(0xFF235B79)],
      ),
      borderRadius: BorderRadius.circular(24),
    ),
    child: Row(
      children: [
        SizedBox(
          height: 92,
          width: 92,
          child: Stack(
            fit: StackFit.expand,
            children: [
              CircularProgressIndicator(
                value: .82,
                strokeWidth: 8,
                strokeCap: StrokeCap.round,
                backgroundColor: Colors.white24,
                valueColor: const AlwaysStoppedAnimation(AppColors.primary),
              ),
              const Center(
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Text(
                      '82',
                      style: TextStyle(
                        color: Colors.white,
                        fontWeight: FontWeight.w800,
                        fontSize: 27,
                      ),
                    ),
                    Text(
                      'score',
                      style: TextStyle(color: Colors.white70, fontSize: 11),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
        const SizedBox(width: 18),
        const Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Your health snapshot',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 18,
                  fontWeight: FontWeight.w800,
                ),
              ),
              SizedBox(height: 6),
```

```
              Text(
                "You're doing well. One habit needs attention today.",
                style: TextStyle(color: Colors.white70, height: 1.35),
              ),
              SizedBox(height: 10),
              Text(
                'VIEW INSIGHTS',
                style: TextStyle(
                  color: Color(0xFFFFD87B),
                  fontSize: 12,
                  fontWeight: FontWeight.w800,
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  );
}

class _CarePlanRow extends StatelessWidget {
  const _CarePlanRow({
    required this.icon,
    required this.color,
    required this.title,
    required this.subtitle,
    required this.trailing,
  });
  final IconData icon;
  final Color color;
  final String title;
  final String subtitle;
  final String trailing;
  @override
  Widget build(BuildContext context) => Row(
    children: [
      Container(
        padding: const EdgeInsets.all(10),
        decoration: BoxDecoration(
          color: color.withValues(alpha: .12),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Icon(icon, color: color),
      ),
      const SizedBox(width: 12),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(title, style: const TextStyle(fontWeight: FontWeight.w800)),
            const SizedBox(height: 3),
            Text(
              subtitle,
              style: const TextStyle(fontSize: 12, color: AppColors.slate),
            ),
          ],
        ),
      ),
      Text(
        trailing,
        style: TextStyle(
          fontSize: 11,
          color: color,
          fontWeight: FontWeight.w700,
        ),
      ),
    ],
  );
}

class _InsightCard extends StatelessWidget {
  const _InsightCard();
  @override
  Widget build(BuildContext context) => AppCard(
    color: const Color(0xFFFFFBED),
    child: Row(
      children: [
        const Icon(
          Icons.lightbulb_outline_rounded,
          color: AppColors.primary,
          size: 30,
        ),
        const SizedBox(width: 14),
        Expanded(
          child: Text(
            'A short evening walk can help you reach your activity goal.',
            style: Theme.of(context).textTheme.bodyMedium?.copyWith(
              fontWeight: FontWeight.w600,
              height: 1.35,
            ),
          ),
        ),
      ],
    ),
  );
}
```

lib/features/settings/presentation/profile_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/ui/primary_button.dart';

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});
  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _api = ApiClient();
```

```
final _form = GlobalKey<FormState>();
final _date = TextEditingController();
final _height = TextEditingController();
final _weight = TextEditingController();
final _target = TextEditingController();
String _gender = 'Other';
bool _saving = false;
late Future<Map<String, dynamic>> _profile;

@override
void initState() {
  super.initState();
  _profile = _load();
}

Future<Map<String, dynamic>> _load() async {
  final data = Map<String, dynamic>.from(await _api.get('v1/profile') as Map);
  _date.text = '${data['date_of_birth'] ?? ''}';
  _height.text = '${data['height_cm'] ?? ''}';
  _weight.text = '${data['current_weight'] ?? ''}';
  _target.text = '${data['target_weight'] ?? ''}';
  if (['Male', 'Female', 'Other'].contains(data['gender'])) {
    _gender = data['gender'] as String;
  }
  return data;
}

@override
void dispose() {
  _date.dispose();
  _height.dispose();
  _weight.dispose();
  _target.dispose();
  super.dispose();
}

Future<void> _save() async {
  if (!_form.currentState!.validate() || _saving) return;
  setState(() => _saving = true);
  try {
    await _api.patch('v1/profile', {
      'gender': _gender,
      'date_of_birth': _date.text.trim(),
      'height_cm': num.parse(_height.text),
      'current_weight': num.parse(_weight.text),
      'target_weight': num.parse(_target.text),
    });
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(const SnackBar(content: Text('Profile updated.')));
  } on ApiException catch (error) {
    if (mounted) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(error.message)));
    }
  } finally {
    if (mounted) setState(() => _saving = false);
  }
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Account details')),
  body: FutureBuilder<Map<String, dynamic>>(
    future: _profile,
    builder: (context, snapshot) {
      if (snapshot.connectionState != ConnectionState.done) {
        return const Center(child: CircularProgressIndicator());
      }
      if (snapshot.hasError) {
        return Center(
          child: TextButton(
            onPressed: () => setState(() => _profile = _load()),
            child: const Text('Unable to load profile. Try again'),
          ),
        );
      }
      final data = snapshot.data!;
      return SafeArea(
        top: false,
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Form(
            key: _form,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  '${data['firstname']} ${data['lastname']}',
                  style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                    fontWeight: FontWeight.w800,
                  ),
                ),
                Text(
                  '${data['email']}',
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
                const SizedBox(height: 24),
                DropdownButtonFormField<String>(
                  initialValue: _gender,
                  decoration: const InputDecoration(labelText: 'Gender'),
                  items: const ['Male', 'Female', 'Other']
                      .map(
                        (value) => DropdownMenuItem(
                          value: value,
                          child: Text(value),
                        ),
                      )
                      .toList(),
                  onChanged: (value) => setState(() => _gender = value!),
                ),
                const SizedBox(height: 14),
                _field(_date, 'Date of birth (YYYY-MM-DD)'),
                const SizedBox(height: 14),
                _field(_height, 'Height (cm)'),
```

```
                const SizedBox(height: 14),
                _field(_weight, 'Current weight (kg)'),
                const SizedBox(height: 14),
                _field(_target, 'Target weight (kg)'),
                const SizedBox(height: 24),
                PrimaryButton(
                  label: _saving ? 'Saving...' : 'Save changes',
                  onPressed: _saving ? null : _save,
                ),
              ],
            ),
          ),
        ),
      );
    },
  ),
);
Widget _field(TextEditingController controller, String label) =>
    TextFormField(
      controller: controller,
      keyboardType: label.startsWith('Date')
          ? TextInputType.datetime
          : const TextInputType.numberWithOptions(decimal: true),
      decoration: InputDecoration(labelText: label),
      validator: (value) =>
          value == null || value.trim().isEmpty ? '$label is required' : null,
    );
}
```

lib/features/settings/presentation/settings_screen.dart

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/network/api_client.dart';
import '../../../core/ui/app_card.dart';
import '../../auth/data/auth_session_store.dart';
import '../../auth/presentation/login_screen.dart';
import 'profile_screen.dart';

class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});
  @override
  Widget build(BuildContext context) => ListView(
    padding: const EdgeInsets.fromLTRB(20, 12, 20, 32),
    children: [
      AppCard(
        onTap: () => Navigator.of(
          context,
        ).push(MaterialPageRoute(builder: (_) => const ProfileScreen())),
        child: Row(
          children: [
            const CircleAvatar(
              radius: 28,
              backgroundColor: Color(0xFFFFF0D0),
              child: Icon(
                Icons.person_rounded,
                color: AppColors.primary,
                size: 30,
              ),
            ),
            const SizedBox(width: 14),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    'Jakes Alvin',
                    style: Theme.of(context).textTheme.titleMedium?.copyWith(
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                  const SizedBox(height: 3),
                  const Text(
                    'Getting healthier, one step at a time',
                    style: TextStyle(color: AppColors.slate, fontSize: 12),
                  ),
                ],
              ),
            ),
            const Icon(Icons.chevron_right_rounded, color: AppColors.slate),
          ],
        ),
      ),
      const SizedBox(height: 24),
      _SettingsGroup(
        title: 'Care preferences',
        items: [
          _SettingsItem(
            Icons.notifications_none_rounded,
            'Notifications',
            'Appointments, medicine and wellness alerts',
            onTap: () => _showNotice(
              context,
              'Notifications',
              'Notifications are managed through your medicine reminders and appointment schedules.',
            ),
          ),
          _SettingsItem(
            Icons.security_outlined,
            'Privacy & security',
            'Data sharing and account protection',
            onTap: () => _showNotice(
              context,
              'Privacy & security',
              'Your session is protected with an access token. Sign out from this device when you are finished.',
            ),
          ),
          _SettingsItem(
            Icons.language_rounded,
            'Language & region',
            'English - Kenya',
            onTap: () => _showNotice(
              context,
```

```
              'Language & region',
              'English - Kenya is currently selected.',
            ),
          ),
        ],
      ),
      const SizedBox(height: 20),
      _SettingsGroup(
        title: 'Support',
        items: [
          _SettingsItem(
            Icons.help_outline_rounded,
            'Help centre',
            'Find answers and contact support',
            onTap: () => _showNotice(
              context,
              'Help centre',
              'For local development support, verify Apache and MySQL are running in XAMPP.',
            ),
          ),
          _SettingsItem(
            Icons.info_outline_rounded,
            'About AfyaHive',
            'Version 1.0.0',
            onTap: () => _showNotice(
              context,
              'About AfyaHive',
              'AfyaHive connects your health information, care plans, and wellness tools in one secure place.',
            ),
          ),
        ],
      ),
      const SizedBox(height: 20),
      OutlinedButton.icon(
        onPressed: () => _signOut(context),
        icon: const Icon(Icons.logout_rounded),
        label: const Text('Sign out'),
        style: OutlinedButton.styleFrom(
          foregroundColor: AppColors.danger,
          minimumSize: const Size.fromHeight(52),
          side: const BorderSide(color: Color(0xFFFFCDD0)),
        ),
      ),
    ],
  );

  static void _showNotice(BuildContext context, String title, String message) =>
      showDialog<void>(
        context: context,
        builder: (context) => AlertDialog(
          title: Text(title),
          content: Text(message),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Close'),
            ),
          ],
        ),
      );
  static Future<void> _signOut(BuildContext context) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Sign out?'),
        content: const Text(
          'You will need to sign in again to access your health information.',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Cancel'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Sign out'),
          ),
        ],
      ),
    );
    if (confirmed == true && context.mounted) {
      try {
        await ApiClient().post('v1/auth/logout', {});
      } catch (_) {
        // Clearing the local token is still required if the network is unavailable.
      }
      await const AuthSessionStore().clear();
      if (!context.mounted) return;
      Navigator.of(context).pushAndRemoveUntil(
        MaterialPageRoute(builder: (_) => const LoginScreen()),
        (_) => false,
      );
    }
  }
}

class _SettingsGroup extends StatelessWidget {
  const _SettingsGroup({required this.title, required this.items});
  final String title;
  final List<_SettingsItem> items;

  @override
  Widget build(BuildContext context) => Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        title,
        style: Theme.of(
          context,
        ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
      ),
      const SizedBox(height: 10),
      AppCard(
        padding: EdgeInsets.zero,
        child: Column(
          children: List.generate(items.length, (index) {
            final item = items[index];
```

```
            return Column(
              children: [
                ListTile(
                  leading: Container(
                    padding: const EdgeInsets.all(8),
                    decoration: BoxDecoration(
                      color: AppColors.primary.withValues(alpha: .12),
                      borderRadius: BorderRadius.circular(10),
                    ),
                    child: Icon(item.icon, color: AppColors.primary),
                  ),
                  title: Text(
                    item.title,
                    style: const TextStyle(fontWeight: FontWeight.w700),
                  ),
                  subtitle: Text(
                    item.subtitle,
                    style: const TextStyle(fontSize: 12),
                  ),
                  trailing: const Icon(Icons.chevron_right_rounded),
                  onTap: item.onTap,
                ),
                if (index < items.length - 1)
                  const Divider(height: 1, indent: 68),
              ],
            );
          }),
        ),
      ),
    ],
  );
}

class _SettingsItem {
  const _SettingsItem(
    this.icon,
    this.title,
    this.subtitle, {
    required this.onTap,
  });
  final IconData icon;
  final String title;
  final String subtitle;
  final VoidCallback onTap;
}
```

lib/features/shell/presentation/app_shell.dart

```
import 'package:flutter/material.dart';

import '../../home/presentation/home_screen.dart';
import '../../settings/presentation/settings_screen.dart';
import '../../vitals/presentation/vitals_screen.dart';
import '../../common/presentation/resource_list_screen.dart';

class AppShell extends StatefulWidget {
  const AppShell({super.key});
  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  var _index = 0;
  static const _pages = [HomeScreen(), VitalsScreen(), SettingsScreen()];
  static const _titles = ['AfyaHive', 'My vitals', 'Profile & settings'];

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(
      title: Row(
        children: [
          if (_index == 0)
            Hero(
              tag: 'afyahive-logo',
              child: CircleAvatar(
                radius: 16,
                backgroundColor: Colors.transparent,
                child: Image.asset(
                  'assets/images/afyahive_logo.png',
                  errorBuilder: (_, _, _) =>
                      const Icon(Icons.local_hospital_rounded),
                ),
              ),
            )
          else
            IconButton(
              tooltip: 'Go home',
              onPressed: () => setState(() => _index = 0),
              icon: const Icon(Icons.arrow_back_ios_new_rounded, size: 18),
            ),
          const SizedBox(width: 10),
          Text(
            _titles[_index],
            style: const TextStyle(fontWeight: FontWeight.w800),
          ),
        ],
      ),
      actions: [
        IconButton(
          onPressed: () => Navigator.of(context).push(
            MaterialPageRoute(
              builder: (_) => const ResourceListScreen(
                title: 'Notifications',
                route: 'reminders',
                emptyMessage: 'You have no active care notifications.',
                icon: Icons.notifications_none_rounded,
                fields: [
                  ResourceField(
                    'medication_name',
                    'Medicine name',
                    required: true,
                  ),
                  ResourceField(
                    'schedule_time',
```

```
                  'Time (HH:MM:SS)',
                  required: true,
                ),
                ResourceField(
                  'frequency',
                  'Frequency',
                  kind: InputKind.select,
                  options: ['daily', 'weekly', 'as_needed'],
                ),
              ],
            ),
          ),
          tooltip: 'Notifications',
          icon: Badge(child: const Icon(Icons.notifications_none_rounded)),
        ),
      ],
    ),
    body: SafeArea(
      top: false,
      child: IndexedStack(index: _index, children: _pages),
    ),
    bottomNavigationBar: NavigationBar(
      selectedIndex: _index,
      onDestinationSelected: (value) => setState(() => _index = value),
      destinations: const [
        NavigationDestination(
          icon: Icon(Icons.home_outlined),
          selectedIcon: Icon(Icons.home_rounded),
          label: 'Home',
        ),
        NavigationDestination(
          icon: Icon(Icons.monitor_heart_outlined),
          selectedIcon: Icon(Icons.monitor_heart_rounded),
          label: 'Vitals',
        ),
        NavigationDestination(
          icon: Icon(Icons.person_outline_rounded),
          selectedIcon: Icon(Icons.person_rounded),
          label: 'Profile',
        ),
      ],
    ),
  );
}
```

lib/features/vitals/presentation/vitals_screen.dart

```
import 'package:flutter/material.dart';
import '../../../core/network/api_client.dart';
import '../../../core/network/api_exception.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/ui/app_card.dart';
import '../../../core/ui/primary_button.dart';
import '../../../core/ui/section_header.dart';

class VitalsScreen extends StatefulWidget {
  const VitalsScreen({super.key});
  @override
  State<VitalsScreen> createState() => _VitalsScreenState();
}

class _VitalsScreenState extends State<VitalsScreen> {
  final _api = ApiClient();
  String _range = 'Week';
  late Future<List<Vital>> _future;
  @override
  void initState() {
    super.initState();
    _future = _load();
  }

  Future<List<Vital>> _load() async {
    final data =
        await _api.get('v1/vitals&range=${_range.toLowerCase()}') as List;
    return data
        .map((e) => Vital.fromJson(Map<String, dynamic>.from(e as Map)))
        .toList();
  }

  Future<void> _refresh() async => setState(() => _future = _load());
  Future<void> _add([String initialType = 'blood_pressure']) async {
    final added = await showModalBottomSheet<bool>(
      context: context,
      isScrollControlled: true,
      builder: (_) => _AddVital(api: _api, initialType: initialType),
    );
    if (added == true && mounted) {
      await _refresh();
      if (mounted)
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(const SnackBar(content: Text('Vital reading saved.')));
    }
  }

  @override
  Widget build(BuildContext context) => FutureBuilder<List<Vital>>(
    future: _future,
    builder: (context, snapshot) {
      if (snapshot.connectionState != ConnectionState.done)
        return const Center(child: CircularProgressIndicator());
      if (snapshot.hasError)
        return _Error(
          message: snapshot.error is ApiException
              ? (snapshot.error as ApiException).message
              : 'Unable to load vital readings.',
          onRetry: _refresh,
        );
      final readings = snapshot.data!;
      final latest = <String, Vital>{};
      for (final value in readings) {
        latest.putIfAbsent(value.type, () => value);
      }
```

```
      return RefreshIndicator(
        onRefresh: _refresh,
        child: CustomScrollView(
          key: const PageStorageKey('vitals-scroll'),
          slivers: [
            SliverPadding(
              padding: const EdgeInsets.fromLTRB(20, 12, 20, 100),
              sliver: SliverList(
                delegate: SliverChildListDelegate([
                  Text(
                    'Know your numbers',
                    style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                  const SizedBox(height: 6),
                  const Text(
                    'Record and follow your health readings.',
                    style: TextStyle(color: AppColors.slate),
                  ),
                  const SizedBox(height: 18),
                  SegmentedButton<String>(
                    segments: const [
                      ButtonSegment(value: 'Week', label: Text('Week')),
                      ButtonSegment(value: 'Month', label: Text('Month')),
                      ButtonSegment(value: 'Year', label: Text('Year')),
                    ],
                    selected: {_range},
                    onSelectionChanged: (value) => setState(() {
                      _range = value.first;
                      _future = _load();
                    }),
                  ),
                  const SizedBox(height: 20),
                  _BloodPressure(
                    reading: latest['blood_pressure'],
                    onAdd: _add,
                  ),
                  const SizedBox(height: 24),
                  SectionHeader(
                    title: 'Latest measurements',
                    actionLabel: 'Add reading',
                    onAction: _add,
                  ),
                  const SizedBox(height: 12),
                  GridView.count(
                    crossAxisCount: 2,
                    childAspectRatio: 1.18,
                    crossAxisSpacing: 12,
                    mainAxisSpacing: 12,
                    shrinkWrap: true,
                    physics: const NeverScrollableScrollPhysics(),
                    children: [
                      _Metric(
                        icon: Icons.favorite_rounded,
                        color: const Color(0xFFD14C5B),
                        title: 'Heart rate',
                        reading: latest['heart_rate'],
                        unit: 'bpm',
                        onTap: () => _add('heart_rate'),
                      ),
                      _Metric(
                        icon: Icons.water_drop_rounded,
                        color: const Color(0xFF337ACC),
                        title: 'Blood oxygen',
                        reading: latest['blood_oxygen'],
                        unit: '% SpO■',
                        onTap: () => _add('blood_oxygen'),
                      ),
                      _Metric(
                        icon: Icons.thermostat_rounded,
                        color: AppColors.primary,
                        title: 'Temperature',
                        reading: latest['temperature'],
                        unit: '°C',
                        onTap: () => _add('temperature'),
                      ),
                      _Metric(
                        icon: Icons.monitor_weight_outlined,
                        color: const Color(0xFF6C5CC9),
                        title: 'Weight',
                        reading: latest['weight'],
                        unit: 'kg',
                        onTap: () => _add('weight'),
                      ),
                    ],
                  ),
                  const SizedBox(height: 24),
                  const SectionHeader(title: 'Health trend'),
                  const SizedBox(height: 12),
                  _Trend(
                    readings: readings
                        .where((v) => v.type == 'heart_rate')
                        .toList(),
                  ),
                ]),
              ),
            ),
          ],
        ),
      );
    },
  );
}

class Vital {
  const Vital({
    required this.type,
    required this.value,
    required this.secondary,
    required this.unit,
    required this.at,
  });
  final String type;
  final double value;
  final double? secondary;
  final String unit;
```

```
  final DateTime at;
  factory Vital.fromJson(Map<String, dynamic> j) => Vital(
    type: j['type'] as String,
    value: double.parse('${j['value']}'),
    secondary: j['secondary_value'] == null
        ? null
        : double.parse('${j['secondary_value']}'),
    unit: j['unit'] as String,
    at: DateTime.parse('${j['recorded_at']}'),
  );
  String get display => type == 'blood_pressure' && secondary != null
      ? '${_number(value)} / ${_number(secondary!)}'
      : _number(value);
  String get status {
    if (type == 'blood_pressure')
      return value >= 180 || (secondary ?? 0) >= 120 || value < 90
          ? 'Abnormal'
          : value >= 130 || (secondary ?? 0) >= 80
          ? 'Above healthy range'
          : 'Within healthy range';
    if (type == 'heart_rate')
      return value < 60 || value > 100 ? 'Abnormal' : 'Normal';
    if (type == 'blood_oxygen')
      return value < 90
          ? 'Abnormal'
          : value < 95
          ? 'Above healthy range'
          : 'Normal';
    if (type == 'temperature')
      return value < 35 || value >= 38
          ? 'Abnormal'
          : value >= 37.5
          ? 'Above healthy range'
          : 'Normal';
    return 'Recorded';
  }

  Color get color => status == 'Abnormal'
      ? AppColors.danger
      : status == 'Above healthy range'
      ? AppColors.primaryDark
      : AppColors.success;
  static String _number(double n) =>
      n == n.roundToDouble() ? n.toInt().toString() : n.toStringAsFixed(1);
}

class _BloodPressure extends StatelessWidget {
  const _BloodPressure({required this.reading, required this.onAdd});
  final Vital? reading;
  final VoidCallback onAdd;
  @override
  Widget build(BuildContext c) => Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(24),
      gradient: const LinearGradient(
        colors: [Color(0xFF193A5D), Color(0xFF286184)],
      ),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Row(
          children: [
            Icon(Icons.monitor_heart_outlined, color: Color(0xFFFFD87B)),
            SizedBox(width: 8),
            Text(
              'BLOOD PRESSURE',
              style: TextStyle(
                color: Colors.white70,
                fontSize: 12,
                fontWeight: FontWeight.w800,
                letterSpacing: 1,
              ),
            ),
          ],
        ),
        const SizedBox(height: 14),
        Text(
          reading?.display ?? 'No reading',
          style: const TextStyle(
            color: Colors.white,
            fontSize: 34,
            fontWeight: FontWeight.w800,
          ),
        ),
        Text(
          reading == null
              ? 'Tap the status below to add a reading'
              : 'mmHg · Last updated ${_ago(reading!.at)}',
          style: const TextStyle(color: Colors.white70, fontSize: 12),
        ),
        const SizedBox(height: 16),
        InkWell(
          onTap: onAdd,
          borderRadius: BorderRadius.circular(20),
          child: Container(
            padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 7),
            decoration: BoxDecoration(
              color: Colors.white.withValues(alpha: .12),
              borderRadius: BorderRadius.circular(20),
            ),
            child: Text(
              reading?.status ?? 'Add blood pressure',
              style: TextStyle(
                color: reading?.color ?? Colors.white,
                fontWeight: FontWeight.w700,
                fontSize: 12,
              ),
            ),
          ),
        ),
      ],
    ),
  );
}
class _Metric extends StatelessWidget {
```

```
  const _Metric({
    required this.icon,
    required this.color,
    required this.title,
    required this.reading,
    required this.unit,
    required this.onTap,
  });
  final IconData icon;
  final Color color;
  final String title, unit;
  final Vital? reading;
  final VoidCallback onTap;
  @override
  Widget build(BuildContext c) => AppCard(
    onTap: onTap,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Icon(icon, color: color),
        const Spacer(),
        Text(
          title,
          style: const TextStyle(
            fontSize: 12,
            color: AppColors.slate,
            fontWeight: FontWeight.w600,
          ),
        ),
        const SizedBox(height: 3),
        Text(
          reading == null
              ? 'No reading'
              : '${reading!.display} ${reading!.unit}',
          style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 18),
        ),
        const SizedBox(height: 4),
        Text(
          reading?.status ?? 'Tap to add',
          style: TextStyle(
            color: reading?.color ?? AppColors.slate,
            fontSize: 11,
            fontWeight: FontWeight.w700,
          ),
        ),
      ],
    ),
  );
}

class _Trend extends StatelessWidget {
  const _Trend({required this.readings});
  final List<Vital> readings;
  @override
  Widget build(BuildContext c) {
    final points = readings.reversed.take(7).toList();
    return AppCard(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Heart rate · ${points.length} reading${points.length == 1 ? '' : 's'}',
            style: const TextStyle(fontWeight: FontWeight.w700),
          ),
          const SizedBox(height: 18),
          SizedBox(
            height: 76,
            child: points.length < 2
                ? const Center(
                    child: Text(
                      'Add two heart-rate readings to view a trend.',
                      style: TextStyle(color: AppColors.slate),
                    ),
                  )
                : CustomPaint(
                    painter: _Painter(points.map((e) => e.value).toList()),
                    child: const SizedBox.expand(),
                  ),
          ),
          const SizedBox(height: 6),
          const Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text(
                'Earlier',
                style: TextStyle(color: AppColors.slate, fontSize: 11),
              ),
              Text(
                'Latest',
                style: TextStyle(color: AppColors.slate, fontSize: 11),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

class _Painter extends CustomPainter {
  const _Painter(this.values);
  final List<double> values;
  @override
  void paint(Canvas c, Size s) {
    final min = values.reduce((a, b) => a < b ? a : b),
        max = values.reduce((a, b) => a > b ? a : b),
        delta = max - min == 0 ? 1 : max - min,
        path = Path();
    for (var i = 0; i < values.length; i++) {
      final x = s.width * i / (values.length - 1),
          y = s.height - ((values[i] - min) / delta) * (s.height - 8) - 4;
      i == 0 ? path.moveTo(x, y) : path.lineTo(x, y);
    }
    c.drawPath(
      path,
      Paint()
        ..color = const Color(0xFFD14C5B)
```

```
        ..strokeWidth = 3
        ..style = PaintingStyle.stroke
        ..strokeCap = StrokeCap.round,
    );
  }

  @override
  bool shouldRepaint(covariant _Painter old) => old.values != values;
}

class _AddVital extends StatefulWidget {
  const _AddVital({required this.api, required this.initialType});
  final ApiClient api;
  final String initialType;
  @override
  State<_AddVital> createState() => _AddVitalState();
}

class _AddVitalState extends State<_AddVital> {
  final _form = GlobalKey<FormState>();
  final _value = TextEditingController();
  final _second = TextEditingController();
  late String _type;
  bool _saving = false;
  @override
  void initState() {
    super.initState();
    _type = widget.initialType;
  }

  @override
  void dispose() {
    _value.dispose();
    _second.dispose();
    super.dispose();
  }

  String get unit => switch (_type) {
    'blood_pressure' => 'mmHg',
    'heart_rate' => 'bpm',
    'blood_oxygen' => '% SpO■',
    'temperature' => '°C',
    'weight' => 'kg',
    _ => '',
  };
  Future<void> save() async {
    if (!_form.currentState!.validate() || _saving) return;
    setState(() => _saving = true);
    try {
      await widget.api.post('v1/vitals', {
        'type': _type,
        'value': num.parse(_value.text),
        'secondary_value': _type == 'blood_pressure'
            ? num.parse(_second.text)
            : null,
        'unit': unit,
        'recorded_at': DateTime.now()
            .toUtc()
            .toIso8601String()
            .substring(0, 19)
            .replaceAll('T', ' '),
        'source': 'manual',
      });
      if (mounted) Navigator.pop(context, true);
    } on ApiException catch (e) {
      if (mounted)
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(e.message)));
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  @override
  Widget build(BuildContext c) => Padding(
    padding: EdgeInsets.fromLTRB(
      20,
      20,
      20,
      MediaQuery.viewInsetsOf(c).bottom + 20,
    ),
    child: SafeArea(
      top: false,
      child: SingleChildScrollView(
        child: Form(
          key: _form,
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Add vital reading',
                style: Theme.of(
                  c,
                ).textTheme.titleLarge?.copyWith(fontWeight: FontWeight.w800),
              ),
              const SizedBox(height: 16),
              DropdownButtonFormField<String>(
                initialValue: _type,
                decoration: const InputDecoration(labelText: 'Vital type'),
                items:
                    const [
                          ('blood_pressure', 'Blood pressure'),
                          ('heart_rate', 'Heart rate'),
                          ('blood_oxygen', 'Blood oxygen / SpO■'),
                          ('temperature', 'Body temperature'),
                          ('weight', 'Weight'),
                        ]
                        .map(
                          (e) =>
                              DropdownMenuItem(value: e.$1, child: Text(e.$2)),
                        )
                        .toList(),
                onChanged: (v) => setState(() => _type = v!),
              ),
              const SizedBox(height: 12),
```

```
              TextFormField(
                controller: _value,
                keyboardType: const TextInputType.numberWithOptions(
                  decimal: true,
                ),
                decoration: InputDecoration(
                  labelText: _type == 'blood_pressure'
                      ? 'Systolic (top number)'
                      : 'Value ($unit)',
                ),
                validator: (v) => v != null && num.tryParse(v) != null
                    ? null
                    : 'Enter a valid number',
              ),
              if (_type == 'blood_pressure') ...[
                const SizedBox(height: 12),
                TextFormField(
                  controller: _second,
                  keyboardType: const TextInputType.numberWithOptions(
                    decimal: true,
                  ),
                  decoration: const InputDecoration(
                    labelText: 'Diastolic (bottom number)',
                  ),
                  validator: (v) => v != null && num.tryParse(v) != null
                      ? null
                      : 'Enter a valid number',
                ),
              ],
              const SizedBox(height: 20),
              PrimaryButton(
                label: _saving ? 'Saving...' : 'Save reading',
                onPressed: _saving ? null : save,
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

String _ago(DateTime d) {
  final x = DateTime.now().difference(d.toLocal());
  if (x.inMinutes < 1) return 'just now';
  if (x.inHours < 1) return '${x.inMinutes}m ago';
  if (x.inDays < 1) return '${x.inHours}h ago';
  return '${x.inDays}d ago';
}

class _Error extends StatelessWidget {
  const _Error({required this.message, required this.onRetry});
  final String message;
  final Future<void> Function() onRetry;
  @override
  Widget build(BuildContext c) => Center(
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text(message),
        OutlinedButton(onPressed: onRetry, child: const Text('Try again')),
      ],
    ),
  );
}
```

lib/main.dart

```
import 'package:flutter/material.dart';

import 'app/afya_hive_app.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const AfyaHiveApp());
}
```

test/widget_test.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

import 'package:frontend_hive/features/auth/presentation/login_screen.dart';

void main() {
  testWidgets('shows the AfyaHive sign-in experience', (tester) async {
    await tester.pumpWidget(const MaterialApp(home: LoginScreen()));

    expect(find.text('Your health,\nall in one place.'), findsOneWidget);
    expect(find.text('Secure Sign In'), findsOneWidget);
  });
}
```